CSCI 4061 - Lab 6

C Extensions for Python

October 19, 2026



UNIVERSITY OF MINNESOTA
Driven to Discover®

Python Extensions (Why?)

- When we want to write C code **specifically** for use from Python
  - Write a C extension for Python
  - Must adhere to specific API conventions

- Typical Pattern for an Extension Function:
  - Convert passed in Python objects to C data types (unboxing)
  - Do work
  - Return a Python object constructed from a C value (boxing)

Python Extension Functions

Every C extension called from Python has a specific form:

- Two arguments of type PyObject *
  - First argument has module data
  - Second argument contains all function arguments as a tuple
  - **Python.h** provides functions to convert arguments to C data types
- Return type of PyObject *
  - Return NULL: Exception is raised by interpreter
  - Otherwise: **Python.h** provides functions to assemble new Python objects

Python Extension Function Declaration

The **Python.h** header file is ***REQUIRED***

```
#include <Python.h>
#include <stdio.h>

PyObject *message(PyObject *self, PyObject *args)
```

```
PyObject *inc_list(PyObject *self, PyObject *args);
```

Parsing Arguments

General Form Is:

```
PyArg_ParseTuple(args, <format string>, ptrs))
```

Format strings are similar to `struct` packing, but more powerful

Format String	C Data Type
i	int
I	unsigned int
l	long
s	char *
f	Float
O	Generic Object
O!	Object of Specific Type





Parsing Arguments Example

```c
PyObject *message(PyObject *self, PyObject *args) {
    long num;
    if (!PyArg_ParseTuple(args, "l", &num)) {
        return NULL;
    }
```

```c
PyObject *inc_list(PyObject *self, PyObject *args) {
    PyObject *input_list = NULL;
    if (!PyArg_ParseTuple(args, "O!", &PyList_Type, &input_list)) {
        // Invalid number of arguments, or argument was not a list
        // Proper exception will be raised for us
        return NULL;
    }
```

Reference Counting

Every PyObject struct has a refcnt field

- refcnt: number of references to the object still in use
- Safe to free PyObject when refcnt reaches 0

Python objects passed into extension functions have positive refcnt

- We're just using these objects temporarily, don't need to worry about how they get cleaned up
- So no need to mess with reference counts

Reference Counting

What about new Python objects we create inside of C?

- PyList_New returns new object with reference count 1
- Our **C code** is considered the **owner** of this reference until list is returned

What if we get an error and don't want to return the list?

- Py_DECREF(my_list) – prevents memory leak once C extension function returns

Reference Counting (w/ Error Checking)

Creating a New List (With Error Checking)

```c
PyObject *output_list = PyList_New(list_len);
if (output_list == NULL) {
    return PyErr_NoMemory();
}
```

```c
// Increment each list element
for (int i = 0; i < list_len; i++) {
    PyObject *current = PyList_GetItem(input_list, i);
    if (!PyLong_Check(current)) {
        PyErr_Format(PyExc_TypeError, "List item %d is not a number", i);
        Py_DECREF(output_list);
        return NULL;
    }
```

If an Error Occurs Later:

```c
if (val < 0) {
    PyErr_SetString(PyExc_ValueError, "Negative values not allowed");
    Py_DECREF(output_list);
    return NULL;
}
```

Showing Errors

The Python interpreter will raise an exception if a C extension function returns NULL

C code must specify which exception to raise before returning

Setting exception type and **message**:

PyErr_SetString(PyExc_ValueError, “Value out of bounds”);

Setting exception type and **message with formatting**:

PyErr_Format(PyExc_ValueError, “%d is an invalid number”, i);

Using value of errno set by a failed C system call to determine msg: PyErr_SetFromErrno(PyExc_OSError);

Check Python Type in C Code

PyLong_Check
PyFloat_Check
PyUnicode_Check
PyTuple_Check
PyList_Check
PyDict_Check
etc.

```
// Increment each list element
for (int i = 0; i < list_len; i++) {
    PyObject *current = PyList_GetItem(input_list, i);
    if (!PyLong_Check(current)) {
```

C into Python & Python into C

```
PyObject *object1 = PyLong_FromLong(4061);
long result1 = PyLong_AsLong(object1);
PyObject *object2 = PyFloat_FromDouble(40.61);
double result2 = PyFloat_AsDouble(object2);

PyObject *object = Py_BuildValue("(ll)", 2021, 4061);
PyObject *object = PyUnicode_FromString("Hello, World!");
```

- Copies bytes of string into a Python object, so the C code still owns this memory

Accepting a List Argument:

```
PyObject *arg_list = NULL;
PyArg_ParseTuple(args, "O!", &PyList_Type, &arg_list);
```

Get List Length:

```
int list_len = PyList_Size(input_list);
```

Getting List Elements:

```
PyObject *current = PyList_GetItem(arg_list, 4);
```

Making a New List:

```
PyObject *my_list = PyList_New(list_len);
```

Setting List Elements:

```
PyObject *current = PyList_SetItem(my_list, index, value);
```

